

🔗 See Topic 03 → Same Direction Two Pointers Foundation

💡 **AHA MOMENT #1: SLIDING WINDOW AS A TWO-POINTER EXTENSION**

Sliding Window is essentially an extension of **Same Direction Two Pointers**! Instead of recomputing sum/counts from scratch for every subarray (taking $O(N \cdot K)$ or $O(N^2)$), we maintain a contiguous "window" state, adding `right` element on expansion and subtracting `left` element on contraction in $O(1)$ amortized per element!

🚀 PATTERN EVOLUTION: HOW ALGORITHMS EVOLVE

Nested Loops $O(N^2)$ → Two Pointers $O(N)$ → Sliding Window $O(N)$ → Prefix Sum $O(1)$ Query

- **Nested Loops:** Recomputes every contiguous subarray sum from scratch.
- **Two Pointers:** Prunes outer pair bounds using sorted arrays or inward criteria.
- **Sliding Window:** Maintains dynamic contiguous bounds, updating edge state incrementally in $O(1)$!

📊 PATTERN COMPARISON & TRIGGER MATRIX

Problem Phrase	Chosen Pattern	Pointer Movement	Target Complexity
"Sorted Pair Target Sum"	Opposite Two Pointers	<code>L = 0, R = N - 1</code>	Time: <code>O(N)</code> , Space: <code>O(1)</code>
"In-Place Remove Element"	Same Direction Pointers	<code>writer = 0, reader = 0</code>	Time: <code>O(N)</code> , Space: <code>O(1)</code>
"Contiguous Subarray Size K"	Fixed Sliding Window	Maintain size K, slide L & R together	Time: <code>O(N)</code> , Space: <code>O(1)</code>
"Longest Substring / Min Window"	Variable Sliding Window	Expand R, shrink L on condition	Time: <code>O(N)</code> , Space: <code>O(1)</code>

🌟 **KEY TAKEAWAY:** Sliding Window eliminates redundant inner loops by updating window state incrementally at array edges!

💡 AHA MOMENT #2: ROLLING SUM FORMULA

For a fixed window of size K , when moving from window $[i, i+K-1]$ to $[i+1, i+K]$, the new sum is simply:

$$\text{newSum} = \text{oldSum} + \text{nums}[i + K] - \text{nums}[i]$$
(Add incoming right element, subtract outgoing left element)!

1 FIXED WINDOW CORE CONCEPT

First initialize the sum/state of the first K elements. Then loop `right` from index K to $N-1$, adding `nums[right]` and subtracting `nums[right - K]` in $O(1)$ time per step.

2 FIXED WINDOW JAVA TEMPLATE

```
public int fixedWindowTemplate(int[] nums, int k) {
    int windowSum = 0;
    // Step 1: Build first window of size K
    for (int i = 0; i < k; i++) windowSum += nums[i];
    int maxSum = windowSum;

    // Step 2: Slide window from K to N-1
    for (int right = k; right < nums.length; right++) {
        windowSum += nums[right] - nums[right - k]; // Rolling sum
        maxSum = Math.max(maxSum, windowSum);
    }
    return maxSum;
}
```

🧠 STEP-BY-STEP WINDOW EVOLUTION VISUALIZATION

Input: `nums = [2, 1, 5, 1, 3, 2]`, `K = 3`

INITIAL WINDOW (`i=0` to `2`): `[2, 1, 5]`

Sum = $2 + 1 + 5 = 8$ | MaxSum = 8

SLIDE 1 (`right=3`, `outgoing=nums[0]=2`, `incoming=nums[3]=1`): `[1, 5, 1]`

Sum = $8 + 1 - 2 = 7$ | MaxSum = 8

SLIDE 2 (`right=4`, `outgoing=nums[1]=1`, `incoming=nums[4]=3`): `[5, 1, 3]`

Sum = $7 + 3 - 1 = 9$ | MaxSum = 9 (NEW MAX!)

⚠️ COMMON BUG: Off-by-one error when subtracting outgoing element! Outgoing index is `right - K`, not `right - K + 1`.

💡 AHA MOMENT #3: EXPAND UNTIL INVALID, SHRINK UNTIL VALID

In Variable Windows, outer `right` pointer EXPANDS the window to include new elements. When a constraint is violated, inner `while` loop advances `left` pointer to SHRINK the window until validity is restored!

1 VARIABLE WINDOW MECHANICS

Both `left` and `right` start at 0. Outer `for` loop moves `right` forward. If window condition becomes invalid, inner `while` loop increments `left` and removes `nums[left]` from state.

2 VARIABLE WINDOW JAVA TEMPLATE

```
public int variableWindowTemplate(int[] nums, int target) {
    int left = 0, currentSum = 0, maxLen = 0;
    for (int right = 0; right < nums.length; right++) {
        currentSum += nums[right]; // Step 1: Expand window
        // Step 2: Shrink while window condition is invalid
        while (currentSum > target) {
            currentSum -= nums[left];
            left++;
        }
        // Step 3: Record valid window max length
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

🧩 WINDOW EVOLUTION: LONGEST SUBARRAY SUM ≤ 7

Input: `nums = [2, 3, 1, 2, 4, 3]`, Target = `7`

EXPAND 1 (right=0..2): Window [2, 3, 1]

Sum = 6 ≤ 7 (Valid) | Length = 3 - 0 = 3

EXPAND 2 (right=3): Window [2, 3, 1, 2]

Sum = 8 > 7 (INVALID!) → Shrink left=0 (sub 2): Window [3, 1, 2], Sum = 6 ≤ 7 (Valid!) | Length = 3

EXPAND 3 (right=4): Window [3, 1, 2, 4]

Sum = 10 > 7 (INVALID!) → Shrink left=1 (sub 3): Window [1, 2, 4], Sum = 7 ≤ 7 (Valid!) | Length = 3

★ KEY TAKEAWAY: Variable Window processes N elements in $O(N)$ total time because `left` and `right` move forward at most N times!

1 CHARACTER FREQUENCY ARRAY VS HASHMAP

Primitive int[128] Array Optimization:

When dealing with ASCII characters, use `int[128] freq` instead of `HashMap<Character, Integer>`! It eliminates object boxing overhead and runs 10x faster in FAANG benchmarks!

```
// Fast ASCII Frequency Tracking
int[] freq = new int[128];
freq[s.charAt(right)]++; // Expand char
// Shrink char:
freq[s.charAt(left)]--; left++;
```

2 AHA MOMENT: "EXACTLY K" TO "AT MOST K" TRICK

💡 THE EXACTLY K FORMULA:

Directly counting subarrays with *Exactly K* distinct elements is complex because sliding windows naturally bound range ceilings. Use the universal mathematical identity:

$$\text{Exactly}(K) = \text{AtMost}(K) - \text{AtMost}(K - 1)!$$

This simple subtraction converts hard "Exactly K" problems (like LC 992) into two standard "At Most K" variable window passes!

🔧 JAVA SLIDING WINDOW TOOLKIT & HACKS

Valid Match Counter Trick:

```
// Track matching unique character counts
int matches = 0;
if (map.get(c) == targetMap.get(c)) matches++;
```

Window Length Calculation:

```
// Current window length between left & right inclusive
int windowLen = right - left + 1;
```

🌟 **KEY TAKEAWAY:** `int[128]` array frequency lookup is $O(1)$ constant time with zero garbage collection pause!

RECOGNITION SIGNALS: KEYWORD TO WINDOW CHOICE

Problem Trigger Keyword	Sliding Window Pattern Choice	Window Condition Rule	Target Complexity
"Contiguous subarray of size K"	Fixed Window (Size K)	Maintain fixed window size \$K\$, slide `left` & `right` together.	Time: <code>O(N)</code> , Space: <code>O(1)</code>
"Longest substring without repeating"	Variable Window (Longest)	Expand `right`. Shrink `left` while `freq[char] > 1`.	Time: <code>O(N)</code> , Space: <code>O(1)</code>
"Minimum window substring containing T"	Variable Window (Shortest)	Expand `right` until valid. Shrink `left` aggressively while valid.	Time: <code>O(N)</code> , Space: <code>O(1)</code>
"Subarrays with exactly K distinct"	At Most K Math Formula	Return <code>AtMost(K) - AtMost(K - 1)</code> .	Time: <code>O(N)</code> , Space: <code>O(K)</code>

SLIDING WINDOW ARCHITECTURAL DECISION CARDS

Fixed Window

Window size \$K\$ given explicitly in prompt. Single loop pass with rolling sum/count.

Variable (Longest)

Find max window length. Expand `right`, shrink `left` when condition is violated.

Variable (Shortest)

Find min window length. Expand `right` until valid, shrink `left` aggressively.



FAANG INTERVIEW TIP: State whether the problem requires a Fixed or Variable window before writing code!

1. LEETCODE 121 — BEST TIME TO BUY & SELL STOCK

 **Easy** ★★★★★ **Dynamic Window** [Amazon](#) [Meta](#)

★ **FLOWCHART:** Read Prompt → Single Buy/Sell Pair → Track Min Buying Price → Window Profit $O(N)$

Problem:

Find max profit from buying on day `i` and selling on day `j > i`.

Example:

prices = [7,1,5,3,6,4]

→

5

?

Why Not DP $O(N^2)$? Nested loops checking all buy/sell pairs TLE. Tracking `minPrice` dynamically achieves $O(N)$ time & $O(1)$ space!

Interviewer: Can you do this in one pass?

Candidate: Yes! Maintain `minPrice` seen so far and calculate profit at each day in $O(N)$.

```
public int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE, maxProfit = 0;
    for (int price : prices) {
        if (price < minPrice) minPrice = price; // Step 1: Update
        min buying price
        else maxProfit = Math.max(maxProfit, price - minPrice); //
        Step 2: Calculate max profit
    }
    return maxProfit;
}
```

2. LEETCODE 643 — MAXIMUM AVERAGE SUBARRAY I

 **Easy** ★★★★★ **Fixed Window K** [Google](#) [MS](#)

★ **FLOWCHART:** Read Prompt → Subarray Size K Given → Fixed Window Rolling Sum $O(N)$

Problem:

Max average of contiguous subarray of length

k

.

Example:

nums = [1,12,-5,-6,50,3], k = 4

→

12.75

?

Why Not Recompute Sum $O(N \cdot K)$? Sliding window adds `nums[i]` and subtracts `nums[i-k]` in $O(1)$ per step!

```
public double findMaxAverage(int[] nums, int k) {
    double sum = 0;
    for (int i = 0; i < k; i++) sum += nums[i]; // Step 1: Build
    first window
    double maxSum = sum;
    for (int i = k; i < nums.length; i++) {
        sum += nums[i] - nums[i - k]; // Step 2: Rolling sum update
        maxSum = Math.max(maxSum, sum);
    }
    return maxSum / k;
}
```

💡 **AHA MOMENT:** Fixed Window eliminates redundant element additions by using `sum += incoming - outgoing`!



3. LEETCODE 3 — LONGEST SUBSTRING WITHOUT REPEATING CHARS

Medium ★★★★★ Variable Window Meta Google Amazon

★ FLOWCHART: Prompt "No Repeats" → Track Last Seen Index → Jump Left Pointer O(N)

Problem:

Length of longest substring without repeating chars.

Example:

s = "abcabcbb"

→

3

("abc").

❓ **Why Not HashSet O(N) Shrink?** 'int[128] lastSeen' array lets 'left' JUMP directly past duplicates in O(1)!**Interviewer:** How to avoid slow character-by-character left pointer shrinking?**Candidate:** Store last seen index of each character to jump 'left' instantly!

```
public int lengthOfLongestSubstring(String s) {
    int[] lastSeen = new int[128];
    Arrays.fill(lastSeen, -1);
    int left = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (lastSeen[c] ≥ left) left = lastSeen[c] + 1; // Step 1:
        Jump left
        lastSeen[c] = right; // Step 2: Update last seen position
        maxLen = Math.max(maxLen, right - left + 1); // Step 3:
        Track max len
    }
    return maxLen;
}
```

4. LEETCODE 424 — LONGEST REPEATING CHARACTER REPLACEMENT

Medium ★★★★★ Variable Frequency Google Uber

★ FLOWCHART: Prompt "K Replacements" → Track maxFreq → Valid if (len - maxFreq ≤ k)

Problem:

Longest substring of same char after replacing at most

k

chars.

Example:

s = "AABABBA", k = 1

→

4

❓ **Why Not Recalculate maxFreq on Shrink?** 'maxFreq' only needs to track global max because window length only increases when 'maxFreq' grows!

```
public int characterReplacement(String s, int k) {
    int[] counts = new int[26];
    int left = 0, maxFreq = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        maxFreq = Math.max(maxFreq, ++counts[s.charAt(right) -
        'A']);
        if ((right - left + 1) - maxFreq > k) { // Step 1: Shrink
            if invalid
                counts[s.charAt(left) - 'A']--;
                left++;
        }
        maxLen = Math.max(maxLen, right - left + 1); // Step 2:
        Record max len
    }
    return maxLen;
}
```

💡 **AHA MOMENT (LC 3):** Using 'int[128] lastSeen' array allows 'left' pointer to JUMP directly past the last duplicate position!

5. LEETCODE 567 — PERMUTATION IN STRING

🟡 Medium ★★★★★ Fixed Window `s1.length()` [Meta](#) [Datadog](#)

🔗 FLOWCHART: Prompt "Permutation Match" → Fixed Window size `s1.length()` → Compare `int[26]` Arrays

Problem:

Return

`true`

if

`s2`

contains a permutation of

`s1`

.

Example:

`s1 = "ab", s2 = "eidbaooo"`

→

`true`

("ba").

❓ Why Not Sort Substrings $O(N \cdot K \log K)$? Sliding fixed frequency map compares in $O(26) = O(1)$ time per window!

```
public boolean checkInclusion(String s1, String s2) {
    if (s1.length() > s2.length()) return false;
    int[] s1Map = new int[26], s2Map = new int[26];
    for (int i = 0; i < s1.length(); i++) {
        s1Map[s1.charAt(i) - 'a']++; s2Map[s2.charAt(i) - 'a']++;
    }
    if (Arrays.equals(s1Map, s2Map)) return true;
    for (int i = s1.length(); i < s2.length(); i++) {
        s2Map[s2.charAt(i) - 'a']++; // Step 1: Add right
        s2Map[s2.charAt(i - s1.length()) - 'a']--; // Step 2: Sub
    }
    if (Arrays.equals(s1Map, s2Map)) return true;
    return false;
}
```

6. LEETCODE 438 — FIND ALL ANAGRAMS IN A STRING

🟡 Medium ★★★★★ Fixed Window `p.length()` [Amazon](#) [Apple](#)

🔗 FLOWCHART: Prompt "All Anagram Indices" → Fixed Window size `p.length()` → Record match indices

Problem:

Return all start indices of

`p`

's anagrams in

`s`

.

Example:

`s = "cbaebabacd", p = "abc"`

→

`[0, 6]`

.

```
public List<Integer> findAnagrams(String s, String p) {
    List<Integer> res = new ArrayList<>();
    if (s.length() < p.length()) return res;
    int[] pCount = new int[26], sCount = new int[26];
    for (int i = 0; i < p.length(); i++) {
        pCount[p.charAt(i) - 'a']++; sCount[s.charAt(i) - 'a']++;
    }
    if (Arrays.equals(pCount, sCount)) res.add(0);
    for (int i = p.length(); i < s.length(); i++) {
        sCount[s.charAt(i) - 'a']++;
        sCount[s.charAt(i - p.length()) - 'a']--;
        if (Arrays.equals(pCount, sCount)) res.add(i - p.length() + 1);
    }
    return res;
}
```

💡 AHA MOMENT: Permutation in String & Find All Anagrams use identical fixed sliding window mechanisms of size `p.length()`!



7. LEETCODE 904 — FRUIT INTO BASKETS (AT MOST 2 TYPES)

Medium ★★★★★ At Most 2 Distinct Google Uber

★ FLOWCHART: Prompt "2 Baskets" → Map size ≤ 2 → Shrink left when map.size() > 2**Problem:**

Max total fruits collected into at most 2 baskets (2 distinct types).

Example:

fruits = [1,2,3,2,2]

→

4

([2,3,2,2]).

❓ **Why Not DP?** Window validity depends strictly on distinct map keys, solvable in $O(N)$ linear space/time!

```
public int totalFruit(int[] fruits) {
    Map<Integer, Integer> count = new HashMap<>();
    int left = 0, maxFruit = 0;
    for (int right = 0; right < fruits.length; right++) {
        count.put(fruits[right], count.getOrDefault(fruits[right], 0) + 1);
        while (count.size() > 2) {
            count.put(fruits[left], count.get(fruits[left]) - 1);
            if (count.get(fruits[left]) == 0)
                count.remove(fruits[left]);
            left++;
        }
        maxFruit = Math.max(maxFruit, right - left + 1);
    }
    return maxFruit;
}
```

8. LEETCODE 1695 — MAXIMUM ERASURE VALUE

Medium ★★★★★ Unique Subarray Sum Meta Amazon

★ FLOWCHART: Prompt "Unique Subarray Max Sum" → HashSet Unique Check → Shrink left on duplicate

Problem:

Max score from erasing a contiguous subarray containing unique elements.

Example:

nums = [4,2,4,5,6]

→

17

([2,4,5,6]).

```
public int maximumUniqueSubarray(int[] nums) {
    Set<Integer> set = new HashSet<>();
    int left = 0, currentSum = 0, maxSum = 0;
    for (int right = 0; right < nums.length; right++) {
        while (set.contains(nums[right])) {
            set.remove(nums[left]);
            currentSum -= nums[left];
            left++;
        }
        set.add(nums[right]);
        currentSum += nums[right];
        maxSum = Math.max(maxSum, currentSum);
    }
    return maxSum;
}
```

💡 **AHA MOMENT:** Fruit Into Baskets is simply "Longest Subarray with At Most 2 Distinct Elements"!

9. LEETCODE 76 — MINIMUM WINDOW SUBSTRING

Hard ★★★★★ Shortest Window [Meta](#) [Google](#) [Amazon](#)

🔥 **FLOWCHART:** Prompt "Min Substring T" → Expand until valid → Shrink left aggressively while valid!

Problem:

Minimum window substring in

s

containing all characters of

t

Example:

s = "ADOBECODEBANC", t = "ABC"

→

"BANC"

Interviewer: Why expand right first, then shrink left inside a inner loop?

Candidate: Outer loop finds a valid candidate window; inner loop minimizes window size while maintaining validity!

```
public String minWindow(String s, String t) {
    if (s.length() < t.length()) return "";
    int[] targetMap = new int[128];
    for (char c : t.toCharArray()) targetMap[c]++;
    int left = 0, minLen = Integer.MAX_VALUE, start = 0, required = t.length();
    for (int right = 0; right < s.length(); right++) {
        if (targetMap[s.charAt(right)]-- > 0) required--;
        while (required == 0) { // Step 1: Valid window!
            if (right - left + 1 < minLen) {
                minLen = right - left + 1; start = left; // Step 2:
            }
            Record min
            if (++targetMap[s.charAt(left)] > 0) required++;
            left++; // Step 3: Shrink left
        }
    }
    return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start + minLen);
}
```

10. LEETCODE 992 — SUBARRAYS WITH K DIFFERENT INTEGERS

Hard ★★★★★ At Most K Subtraction [Amazon](#) [Google](#)

🔥 **FLOWCHART:** Prompt "Exactly K Distinct" → Exactly(K) = AtMost(K) - AtMost(K-1)

Problem:

Number of good subarrays with exactly

k

different integers.

```
public int subarraysWithKDistinct(int[] nums, int k) {
    return atMostK(nums, k) - atMostK(nums, k - 1);
}
private int atMostK(int[] nums, int k) {
    Map<Integer, Integer> count = new HashMap<>();
    int left = 0, res = 0;
    for (int right = 0; right < nums.length; right++) {
        count.put(nums[right], count.getOrDefault(nums[right], 0) + 1);
        while (count.size() > k) {
            count.put(nums[left], count.get(nums[left]) - 1);
            if (count.get(nums[left]) == 0) count.remove(nums[left]);
            left++;
        }
        res += right - left + 1; // Step 1: Count all valid ending at right
    }
    return res;
}
```

🧠 **MEMORY BOX:** 'atMost(K)' counts all valid subarrays ending at 'right' with 'right - left + 1' in $O(N)$ time!

 WHEN SLIDING WINDOW FAILS (DO NOT USE!)

1. Negative Numbers in Array (LC 560):

Sliding window relies on **monotonicity** (adding positive numbers increases sum, shrinking left decreases sum). If array has negative numbers, moving 'left' may INCREASE sum! Use **Prefix Sum + HashMap** instead!

2. Non-Contiguous Subsequence Choices (LC 300):

Sliding Window **REQUIRES** contiguous subarrays/substrings. If prompt asks for subsequences or combinations, use **Dynamic Programming / Backtracking!**

 UNSEEN INTERVIEW CHALLENGES (IDENTIFY PATTERN BEFORE LOOKING!)

Challenge Prompt	Pattern Recognition Answer
"Find max sum of non-overlapping 3 subarrays of size K"	Fixed Window + DP Prefix Arrays (Slide 3 fixed size K windows)
"Longest substring with at most K distinct characters"	Variable Window (Longest) (Shrink left while `map.size() > K`)
"Count subarrays with sum equal to K (with negatives)"	WRONG FOR WINDOW! Use Prefix Sum + HashMap!

 **FAANG COACH TIP:** Checking whether negative numbers exist BEFORE choosing Sliding Window is a senior engineer signal!

★ THE 4 GOLDEN LAWS OF SLIDING WINDOW

1. **Fixed Window Size K:** Slide 'left' & 'right' together! `sum += incoming - outgoing`.
2. **Variable Window Longest:** Expand 'right'. Shrink 'left' while condition is INVALID. Update max length.
3. **Variable Window Shortest:** Expand 'right' until valid. Shrink 'left' aggressively while VALID. Update min length.
4. **Exactly K Formula:** `Exactly(K) = AtMost(K) - AtMost(K - 1)!`

🔥 1-MINUTE REVISION CODE TEMPLATE

```
// Universal Variable Window Template
int left = 0, result = 0;
for (int right = 0; right < n; right++) {
    // 1. Expand right
    updateState(nums[right]);
    // 2. Shrink left if invalid
    while (isInvalid()) {
        removeState(nums[left]);
        left++;
    }
    // 3. Update answer
    result = Math.max(result, right - left + 1);
}
```

👉 Next Master Topic: Topic 05 → Binary Search Masterclass

🚩 TOPIC 04 SLIDING WINDOW 10/10 COMPLETE! Ready for Topic 05: Binary Search Masterclass!